

ЛАБОРАТОРНА РОБОТА №2

Породжувальні шаблони

Мета: навчитися реалізовувати породжувальні шаблони проєктування.

Хід роботи:

Завдання 1: Фабричний метод

1. Напишіть систему класів для реалізації функціоналу створення різних типів підписок для відео провайдера.

2. Кожна з підписок повинна мати щомісячну плату, мінімальний період підписки та список каналів й інших можливостей.

3. Види підписок: *DomesticSubscription*, *EducationalSubscription*, *PremiumSubscription*.

4. Придбати (тобто створити) підписку можна за допомогою трьох різних класів: *WebSite*, *MobileApp*, *ManagerCall*, кожен з них має реалізувати свою логіку створення підписок.

5. Покажіть правильність роботи свого коду запустивши його в головному методі програми.

6. Підготуйте діаграму створених у програмі класів та інтерфейсів за допомогою <https://app.diagrams.net/>, експортуйте та завантажте її до репозиторія.

Створюємо абстрактний клас *Subscription* та три його конкретні реалізації: *DomesticSubscription*, *EducationalSubscription* та *PremiumSubscription*.

					ДУ «Житомирська політехніка».26.F2.06.000 – Лр.2			
Змн.	Арк.	№ докум.	Підпис	Дата				
Розроб.		Мазурова І. В.			Звіт з лабораторної роботи		Літ.	Арк.
Перевір.		Левківський В. Л.						1
Керівник								19
Н. контр.							ФІКТ Гр. ВТк-25-1	
Зав. каф.								

Для забезпечення гнучкого процесу генерації об'єктів без жорсткої прив'язки до конкретних типів було застосовано фабричний метод. Створення підписок інкапсульовано у три окремі класи: *WebSite*, *MobileApp* та *ManagerCall*. Кожен із цих класів перевизначає фабричний метод та реалізує індивідуальну бізнес-логіку поведінки: нарахування знижки на сайті, додавання мобільних бонусів у додатку чи додавання сервісного збору при дзвінку до менеджера.

Лістинг Program.cs:

```
namespace FactoryMethod
{
    public abstract class Subscription
    {
        public double MonthlyFee { get; set; }
        public int MinPeriodMonths { get; set; }
        public List<string> Channels { get; set; } = new List<string>();
        public List<string> Features { get; set; } = new List<string>();

        public abstract string GetSubscriptionType();

        public void PrintInfo()
        {
            Console.WriteLine($"--- {GetSubscriptionType()} ---");
            Console.WriteLine($"Щомісячна плата: {MonthlyFee} грн");
            Console.WriteLine($"Мінімальний період: {MinPeriodMonths} міс.");
            Console.WriteLine($"Список каналів: {string.Join(", ", Channels)}");
            Console.WriteLine($"Особливості: {string.Join(", ", Features)}");
            Console.WriteLine(new string('-', 40));
        }
    }

    public class DomesticSubscription : Subscription
    {
        public DomesticSubscription()
        {
            MonthlyFee = 150.0;
            MinPeriodMonths = 1;
            Channels.AddRange(new[] { "UA:Перший", "1+1", "Новий Канал", "ICTV" });
            Features.Add("Базова якість HD");
        }
        public override string GetSubscriptionType() => "DomesticSubscription";
    }

    public class EducationalSubscription : Subscription
```

		Мазурова І. В.			ДУ «Житомирська політехніка».26.F2.06.000 – Лр.2	Арк.
		Левківський В. Л.				2
Змн.	Арк.	№ докум.	Підпис	Дата		

```

{
    public EducationalSubscription()
    {
        MonthlyFee = 120.0;
        MinPeriodMonths = 3;
        Channels.AddRange(new[] { "Discovery Channel", "National Geographic", "Da Vinci" });
        Features.Add("Доступ до наукових тестів");
    }
    public override string GetSubscriptionType() =>
"EducationalSubscription";
}

public class PremiumSubscription : Subscription
{
    public PremiumSubscription()
    {
        MonthlyFee = 400.0;
        MinPeriodMonths = 12;
        Channels.AddRange(new[] { "HBO HD", "Netflix Originals", "Megogo Футбол" });
        Features.Add("Якість 4K Ultra HD");
    }
    public override string GetSubscriptionType() => "PremiumSubscription";
}

public abstract class SubscriptionCreator
{
    public abstract Subscription CreateSubscription(string type);
}

public class WebSite : SubscriptionCreator
{
    public override Subscription CreateSubscription(string type)
    {
        Subscription sub = type.ToLower() switch
        {
            "domestic" => new DomesticSubscription(),
            "educational" => new EducationalSubscription(),
            _ => new PremiumSubscription()
        };

        sub.MonthlyFee *= 0.9;
        sub.Features.Add("Оформлено через WebSite (Знижка 10%)");
        return sub;
    }
}

public class MobileApp : SubscriptionCreator
{

```

```

public override Subscription CreateSubscription(string type)
{
    Subscription sub = type.ToLower() switch
    {
        "domestic" => new DomesticSubscription(),
        "educational" => new EducationalSubscription(),
        _ => new PremiumSubscription()
    };

    sub.Features.Add("Оформлено через MobileApp (Бонусний мобільний трафік)");
    return sub;
}

public class ManagerCall : SubscriptionCreator
{
    public override Subscription CreateSubscription(string type)
    {
        Subscription sub = type.ToLower() switch
        {
            "domestic" => new DomesticSubscription(),
            "educational" => new EducationalSubscription(),
            _ => new PremiumSubscription()
        };

        sub.MonthlyFee += 20;
        sub.Features.Add("Оформлено через дзвінок менеджру (+20 грн за обслуговування)");
        return sub;
    }
}

class Program
{
    static void Main(string[] args)
    {
        Console.OutputEncoding = System.Text.Encoding.UTF8;

        SubscriptionCreator website = new WebSite();
        SubscriptionCreator mobileApp = new MobileApp();
        SubscriptionCreator manager = new ManagerCall();

        Subscription sub1 = website.CreateSubscription("domestic");
        sub1.PrintInfo();

        Subscription sub2 = mobileApp.CreateSubscription("premium");
        sub2.PrintInfo();

        Subscription sub3 = manager.CreateSubscription("educational");
    }
}

```

```

        sub3.PrintInfo();

        Console.ReadKey();
    }
}

```

Результат:

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

Щомісячна плата: 135 грн
Мінімальний період: 1 міс.
Список каналів: UA:Перший, 1+1, Новий Канал, ICTV
Особливості: Базова якість HD, підключай через веб-сайт
-----
--- PremiumSubscription ---
Щомісячна плата: 400 грн
Мінімальний період: 12 міс.
Список каналів: HBO HD, Netflix Originals, Megogo Футбол
Особливості: Якість 4K Ultra HD, підключай через мобільний застосунок (бонусний мобільний трафік)
-----
--- EducationalSubscription ---
Щомісячна плата: 140 грн
Мінімальний період: 3 міс.
Список каналів: Discovery Channel, National Geographic, Da Vinci
Особливості: Доступ до наукових тестів, оформлюй через дзвінок менеджеру
-----
█

```

Рис. 1.1. Результат виконання програми патерну «Фабричний метод»

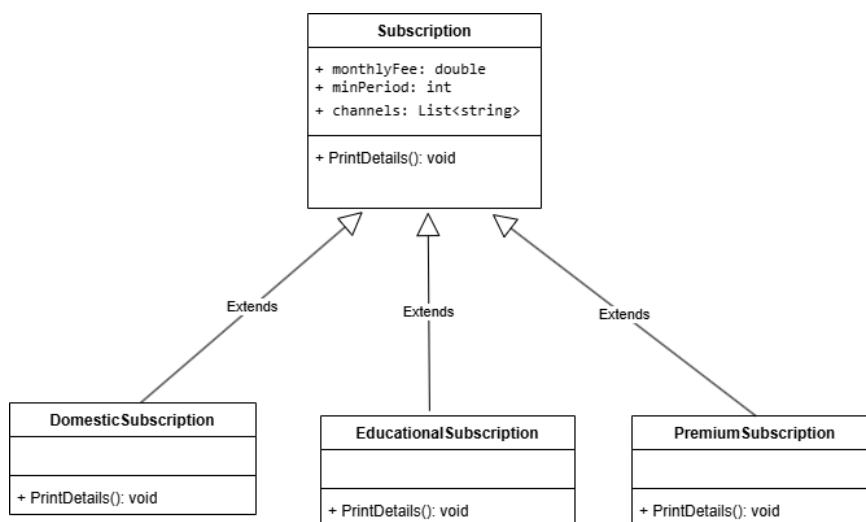


Рис. 1.2. UML-діаграма класів

Завдання 2: Абстрактна фабрика

1. Створіть фабрику виробництва техніки.

2. На фабриці мають створюватися різні девайси (наприклад, Laptop, Netbook, EBook, Smartphone) для різних брендів (IPhone, Kiaomi, Balaxy).

3. Покажіть правильність роботи свого коду запустивши його в головному методі програми.

4. Підготуйте діаграму створених у програмі класів та інтерфейсів за допомогою <https://app.diagrams.net/>, експортуйте та завантажте її до репозиторія.

Створюємо систему для фабрики виробництва техніки різних брендів. Визначено загальні інтерфейси для чотирьох типів девайсів: ноутбуків (ILaptop), нетбуків (INetbook), електронних книг (IEBook) та смартфонів (ISmartphone). Створюємо абстрактний інтерфейс фабрики ITechFactory та три її конкретні реалізації для брендів IPhoneFactory, KiaomiFactory та BalaxyFactory. Кожна конкретна фабрика інкапсулює логіку випуску повної лінійки сумісної продукції у фірмовому стилі свого бренду.

Лістинг Program.cs:

```
namespace AbstractFactory
{
    public interface ILaptop
    {
        void GetInfo();
    }

    public interface INetbook
    {
        void GetInfo();
    }

    public interface IEBook
    {
        void GetInfo();
    }

    public interface ISmartphone
    {
        void GetInfo();
    }

    public class IPhoneLaptop : ILaptop
```

		Мазурова І. В.			ДУ «Житомирська політехніка».26.F2.06.000 – Лр.2	Арк.
		Левківський В. Л.				6
Змн.	Арк.	№ докум.	Підпис	Дата		

```

{
    public void GetInfo() => Console.WriteLine("Ноутбук: IProne Book Pro");
}

public class IProneNetbook : INetbook
{
    public void GetInfo() => Console.WriteLine("Нетбук: IProne Air Mini");
}

public class IProneEBook : IEBook
{
    public void GetInfo() => Console.WriteLine("Електронна книга: IProne Read");
}

public class IProneSmartphone : ISmartphone
{
    public void GetInfo() => Console.WriteLine("Смартфон: IProne 15 Pro");
}

public class KiaomiLaptop : ILaptop
{
    public void GetInfo() => Console.WriteLine("Ноутбук: Kiaomi Mi Notebook");
}

public class KiaomiNetbook : INetbook
{
    public void GetInfo() => Console.WriteLine("Нетбук: Kiaomi Redmi Book Mini");
}

public class KiaomiEBook : IEBook
{
    public void GetInfo() => Console.WriteLine("Електронна книга: Kiaomi Paperwhite");
}

public class KiaomiSmartphone : ISmartphone
{
    public void GetInfo() => Console.WriteLine("Смартфон: Kiaomi 14 Ultra");
}

public class BalaxyLaptop : ILaptop
{
    public void GetInfo() => Console.WriteLine("Ноутбук: Balaxy Book Ultra");
}

public class BalaxyNetbook : INetbook
{
    public void GetInfo() => Console.WriteLine("Нетбук: Balaxy Go Netbook");
}

public class BalaxyEBook : IEBook

```

		Мазурова І. В.			ДУ «Житомирська політехніка».26.F2.06.000 – Лр.2	Арк.
		Левківський В. Л.				7
Змн.	Арк.	№ докум.	Підпис	Дата		

```

{
    public void GetInfo() => Console.WriteLine("Електронна книга: Balaxy Note Read");
}

public class BalaxySmartphone : ISmartphone
{
    public void GetInfo() => Console.WriteLine("Смартфон: Balaxy S26 Ultra");
}

public interface ITechFactory
{
    ILaptop CreateLaptop();
    INetbook CreateNetbook();
    IBook CreateEBook();
    ISmartphone CreateSmartphone();
}

public class IProneFactory : ITechFactory
{
    public ILaptop CreateLaptop() => new IProneLaptop();
    public INetbook CreateNetbook() => new IProneNetbook();
    public IBook CreateEBook() => new IProneEBook();
    public ISmartphone CreateSmartphone() => new IProneSmartphone();
}

public class KiaomiFactory : ITechFactory
{
    public ILaptop CreateLaptop() => new KiaomiLaptop();
    public INetbook CreateNetbook() => new KiaomiNetbook();
    public IBook CreateEBook() => new KiaomiEBook();
    public ISmartphone CreateSmartphone() => new KiaomiSmartphone();
}

public class BalaxyFactory : ITechFactory
{
    public ILaptop CreateLaptop() => new BalaxyLaptop();
    public INetbook CreateNetbook() => new BalaxyNetbook();
    public IBook CreateEBook() => new BalaxyEBook();
    public ISmartphone CreateSmartphone() => new BalaxySmartphone();
}

class Program
{
    static void Main(string[] args)
    {
        Console.OutputEncoding = System.Text.Encoding.UTF8;

        ITechFactory iproneFactory = new IProneFactory();
        Console.WriteLine("--- Виробництво техніки бренду IProne ---");
        ILaptop iproneLaptop = iproneFactory.CreateLaptop();
    }
}

```



```

ISmartphone ipronePhone = iproneFactory.CreateSmartphone();
iproneLaptop.GetInfo();
ipronePhone.GetInfo();
Console.WriteLine(new string('-', 40));

ITechFactory kiaomiFactory = new KiaomiFactory();
Console.WriteLine("--- Виробництво техніки бренду Kiaomi ---");
INetbook kiaomiNetbook = kiaomiFactory.CreateNetbook();
IEBook kiaomiEBook = kiaomiFactory.CreateEBook();
kiaomiNetbook.GetInfo();
kiaomiEBook.GetInfo();
Console.WriteLine(new string('-', 40));

ITechFactory balaxyFactory = new BalaxyFactory();
Console.WriteLine("--- Виробництво техніки бренду Balaxy ---");
ILaptop balaxyLaptop = balaxyFactory.CreateLaptop();
ISmartphone balaxyPhone = balaxyFactory.CreateSmartphone();
balaxyLaptop.GetInfo();
balaxyPhone.GetInfo();
Console.WriteLine(new string('-', 40));

Console.ReadKey();
    }
}
}

```

Результат:

```

--- Виробництво техніки бренду IProne ---
Ноутбук: IProne Book Pro
Смартфон: IProne 15 Pro
-----
--- Виробництво техніки бренду Kiaomi ---
Нетбук: Kiaomi Redmi Book Mini
Електронна книга: Kiaomi Paperwhite
-----
--- Виробництво техніки бренду Balaxy ---
Ноутбук: Balaxy Book Ultra
Смартфон: Balaxy S26 Ultra
-----

```

Рис. 2.1. Результат виконання програми патерну «Абстрактна фабрика»

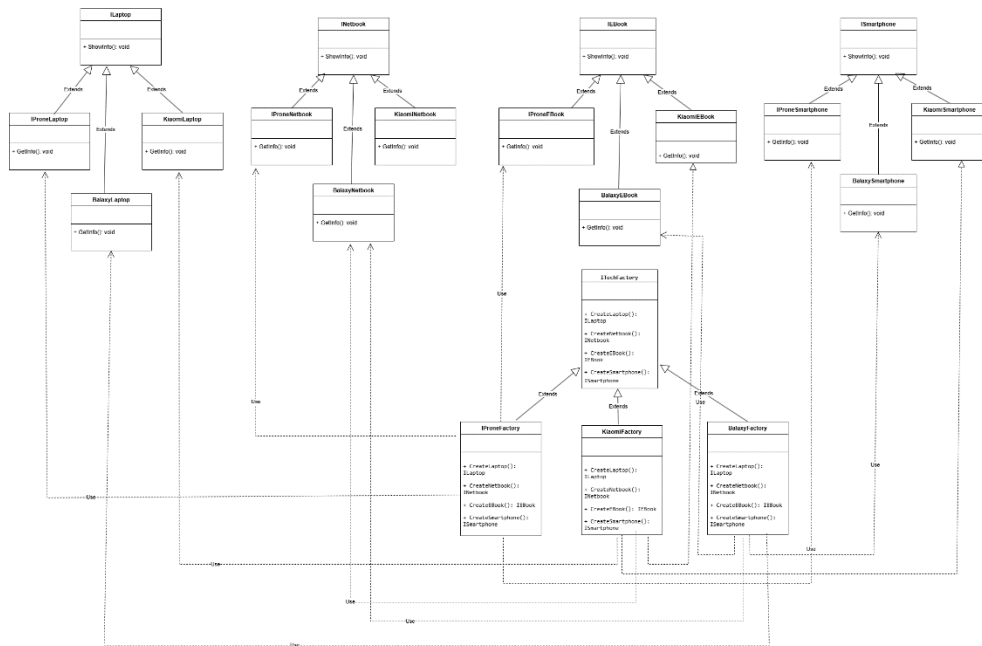


Рис. 2.2. UML-діаграма класів

Завдання 3: Одинак

1. Створіть клас *Authenticator* таким чином, щоб бути впевненим, що цей клас може створити лише один екземпляр, незалежно від кількості потоків і класів, що його наслідують.
2. Покажіть правильність роботи свого коду запустивши його в головному методі програми.

Створюємо клас *Authenticator* за допомогою патерну «Одинак». Головна мета – зробити так, щоб у програмі міг існувати лише один такий екземпляр. Для цього конструктор класу робимо приватним, а сам клас захистили від наслідування словом *sealed*. Щоб об'єкт випадково не створився двічі використано механізм блокування *lock*.

Наприкінці перевірка *ReferenceEquals* вивела *True* як підтвердження, що обидві змінні використовують один і той самий об'єкт у пам'яті.

Лістинг *Program.cs*:

```
namespace SingletonPattern
{
```

```

public sealed class Authenticator
{
    private static Authenticator _instance;
    private static readonly object _lock = new object();

    public string CurrentUser { get; private set; }

    private Authenticator()
    {
        Console.WriteLine("=> Ініціалізація єдиного екземпляра автентифіка-
тора...");
    }

    public static Authenticator Instance
    {
        get
        {
            lock (_lock)
            {
                if (_instance == null)
                {
                    _instance = new Authenticator();
                }
                return _instance;
            }
        }
    }

    public void Login(string username)
    {
        if (CurrentUser == null)
        {
            CurrentUser = username;
            Console.WriteLine($"[Успіх] Користувач {username} увійшов у си-
стему.");
        }
        else
        {
            Console.WriteLine($"[Відмова] Вхід для {username} неможливий.
Вже авторизовано: {CurrentUser}.");
        }
    }
}

class Program
{
    static void Main(string[] args)
    {
        Console.OutputEncoding = System.Text.Encoding.UTF8;
        Console.WriteLine("--- Перевірка патерну Одинак ---");
    }
}

```

```

var auth1 = Authenticator.Instance;
var auth2 = Authenticator.Instance;

auth1.Login("Admin");
auth2.Login("Guest");

Console.Write("auth1 i auth2 посилаються на один об'єкт? ");
Console.WriteLine(ReferenceEquals(auth1, auth2));

Console.ReadKey();
    }
}

```

Результат:

```

--- Перевірка патерну Одинак ---
=> Ініціалізація єдиного екземпляра автентифікатора...
[Успіх] Користувач Admin увійшов у систему.
[Відмова] Вхід для Guest неможливий. Вже авторизовано: Admin.
auth1 i auth2 посилаються на один об'єкт? True

```

Рис. 3.1. Результат виконання програми патерну «Одинак»

Завдання 4: Прототип

1. Створіть клас *Virus*. Він повинен містити вагу, вік, ім'я, вид і масив дітей, екземплярів *Virus*.
2. Створіть екземпляри для цілого “сімейства” вірусів (мінімум три покоління).
3. За допомогою шаблону Прототип реалізуйте можливість клонування наявних вірусів.
4. При клонуванні віруса-батька повинні клонуватися всі його діти.
5. Покажіть правильність роботи свого коду запустивши його в головному методі програми.

Створюємо клас *Virus* з застосуванням патерну «Прототип» який дозволяє створювати точні копії об'єктів. Клас містить поля для збереження ваги, віку, імені, виду та списку дітей. Реалізовуємо метод *Clone()*, який забезпечує «глибоке копіювання». При клонуванні головного вірусу метод рекурсивно створює нові незалежні копії для всіх його нащадків. Щоб довести незалежність створеного клону, в оригінальному екземплярі та його дітях було змінено імена.

Лістинг Program.cs:

```
namespace PrototypePattern
{
    public class Virus
    {
        public double Weight { get; set; }
        public int Age { get; set; }
        public string Name { get; set; }
        public string Species { get; set; }
        public List<Virus> Children { get; set; }

        public Virus(double weight, int age, string name, string species)
        {
            Weight = weight;
            Age = age;
            Name = name;
            Species = species;
            Children = new List<Virus>();
        }

        public Virus Clone()
        {
            Virus clonedVirus = new Virus(this.Weight, this.Age, this.Name,
            this.Species);

            foreach (var child in this.Children)
            {
                clonedVirus.Children.Add(child.Clone());
            }

            return clonedVirus;
        }

        public void PrintInfo(string indent = "")
        {
            Console.WriteLine($"{indent}- {Name} (Вид: {Species}, Вік: {Age},
            Вага: {Weight})");
            foreach (var child in Children)
```

```

        {
            child.PrintInfo(indent + "  ");
        }
    }
}

class Program
{
    static void Main(string[] args)
    {
        Console.OutputEncoding = System.Text.Encoding.UTF8;

        Virus grandchild1 = new Virus(1.5, 1, "Вірус-Онук 1", "Covid-19");
        Virus grandchild2 = new Virus(1.2, 1, "Вірус-Онук 2", "Covid-19");

        Virus child1 = new Virus(2.5, 3, "Вірус-Син", "Covid-19");
        child1.Children.Add(grandchild1);
        child1.Children.Add(grandchild2);

        Virus child2 = new Virus(2.8, 4, "Вірус-Донька", "Covid-19");

        Virus grandParent = new Virus(5.0, 10, "Вірус-дід (Оригінал)",
"Covid-19");
        grandParent.Children.Add(child1);
        grandParent.Children.Add(child2);

        Console.WriteLine("=== Оригінальна сім'я вірусів (3 покоління)
===");
        grandParent.PrintInfo();

        Virus clonedFamily = grandParent.Clone();

        grandParent.Name = "ВІРУС-ДІД (Мутував)";
        grandParent.Children[0].Name = "Вірус-Син (Мутував)";

        Console.WriteLine("\n=== Клоновава сім'я вірусів ===");
        clonedFamily.PrintInfo();

        Console.ReadKey();
    }
}
}

```

Результат:

		Мазурова І. В.			ДУ «Житомирська політехніка».26.F2.06.000 – Лр.2	Арк.
		Левківський В. Л.				14
Змн.	Арк.	№ докум.	Підпис	Дата		

```

=== Оригінальна сім'я вірусів (3 покоління) ===
- Вірус-дід (Оригінал) (Вид: Covid-19, Вік: 10, Вага: 5)
  - Вірус-Син (Вид: Covid-19, Вік: 3, Вага: 2,5)
    - Вірус-Онук 1 (Вид: Covid-19, Вік: 1, Вага: 1,5)
    - Вірус-Онук 2 (Вид: Covid-19, Вік: 1, Вага: 1,2)
  - Вірус-Донька (Вид: Covid-19, Вік: 4, Вага: 2,8)

=== Клоновава сім'я вірусів ===
- Вірус-дід (Оригінал) (Вид: Covid-19, Вік: 10, Вага: 5)
  - Вірус-Син (Вид: Covid-19, Вік: 3, Вага: 2,5)
    - Вірус-Онук 1 (Вид: Covid-19, Вік: 1, Вага: 1,5)
    - Вірус-Онук 2 (Вид: Covid-19, Вік: 1, Вага: 1,2)
  - Вірус-Донька (Вид: Covid-19, Вік: 4, Вага: 2,8)

```

Рис. 4.1. Результат виконання програми патерну «Одинак»

Завдання 5: Будівельник

1. Створіть клас HeroBuilder, який буде створювати персонажа гри, поступово додаючи до нього різні ознаки, наприклад зріст, статуру, колір волосся, очей, одяг, інвентар тощо (можете включити фантазію).

2. Створіть клас EnemyBuilder, який буде реалізовувати єдиний інтерфейс з HeroBuilder. Відмінністю між ними можуть бути спеціальні методи для творення добра або зла, а також списки добрих і злих справ відповідно.

3. За допомогою свого білдера і класу-директора створіть героя (або героїню) своєї мрії 😊, а також свого найзапеклішого ворога.

4. Зверніть увагу, що Ваші білдери повинні реалізовувати текучий інтерфейс (fluent interface).

5. Покажіть правильність роботи свого коду запустивши його в головному методі програми.

6. Підготуйте діаграму створених у програмі класів та інтерфейсів за допомогою <https://app.diagrams.net/>, експортуйте та завантажте її до репозиторія.

У п'ятому завданні реалізовано патерн Будівельник (Builder) для покрокового створення персонажів гри. Створюємо загальний клас *Character*, який містить поля для опису зовнішності, інвентарю та окремі списки для добрих і злих справ. Для

		Мазурова І. В.			ДУ «Житомирська політехніка».26.F2.06.000 – Лр.2	Арк.
		Левківський В. Л.				15
Змн.	Арк.	№ докум.	Підпис	Дата		

конструювання об'єктів розробляємо інтерфейс *ICharacterBuilder* та два конкретні класи: *HeroBuilder* для героя і *EnemyBuilder* для ворога. При виклику загального методу додавання дії, *HeroBuilder* записує її у список добрих справ, а *EnemyBuilder* — у список злих.

Лістинг Program.cs

```
namespace BuilderPattern
{
    public class Character
    {
        public int Height { get; set; }
        public string Build { get; set; }
        public string HairColor { get; set; }
        public string EyeColor { get; set; }
        public string Clothing { get; set; }
        public string Inventory { get; set; }
        public List<string> GoodDeeds { get; set; } = new List<string>();
        public List<string> EvilDeeds { get; set; } = new List<string>();

        public void ShowInfo(string role)
        {
            Console.WriteLine($"--- {role} ---");
            Console.WriteLine($"Зріст: {Height} см, Статура: {Build}");
            Console.WriteLine($"Волосся: {HairColor}, Очі: {EyeColor}");
            Console.WriteLine($"Одяг: {Clothing}");
            Console.WriteLine($"Інвентар: {Inventory}");

            if (GoodDeeds.Count > 0)
                Console.WriteLine($"Добрі справи: {string.Join(", ",
GoodDeeds)}");
            if (EvilDeeds.Count > 0)
                Console.WriteLine($"Злі справи: {string.Join(", ",
EvilDeeds)}");
            Console.WriteLine();
        }
    }

    public interface ICharacterBuilder
    {
        ICharacterBuilder SetHeight(int height);
        ICharacterBuilder SetBuild(string build);
        ICharacterBuilder SetHairColor(string color);
        ICharacterBuilder SetEyeColor(string color);
        ICharacterBuilder SetClothing(string clothing);
        ICharacterBuilder SetInventory(string inventory);
        ICharacterBuilder DoDeed(string deed);
    }
}
```



```

        Character GetResult();
    }

    public class HeroBuilder : ICharacterBuilder
    {
        private Character _character = new Character();

        public ICharacterBuilder SetHeight(int height) { _character.Height = height; return this; }
        public ICharacterBuilder SetBuild(string build) { _character.Build = build; return this; }
        public ICharacterBuilder SetHairColor(string color) { _character.HairColor = color; return this; }
        public ICharacterBuilder SetEyeColor(string color) { _character.EyeColor = color; return this; }
        public ICharacterBuilder SetClothing(string clothing) { _character.Clothing = clothing; return this; }
        public ICharacterBuilder SetInventory(string inventory) { _character.Inventory = inventory; return this; }
        public ICharacterBuilder DoDeed(string deed) { _character.GoodDeeds.Add(deed); return this; }
        public Character GetResult() { return _character; }
    }

    public class EnemyBuilder : ICharacterBuilder
    {
        private Character _character = new Character();

        public ICharacterBuilder SetHeight(int height) { _character.Height = height; return this; }
        public ICharacterBuilder SetBuild(string build) { _character.Build = build; return this; }
        public ICharacterBuilder SetHairColor(string color) { _character.HairColor = color; return this; }
        public ICharacterBuilder SetEyeColor(string color) { _character.EyeColor = color; return this; }
        public ICharacterBuilder SetClothing(string clothing) { _character.Clothing = clothing; return this; }
        public ICharacterBuilder SetInventory(string inventory) { _character.Inventory = inventory; return this; }
        public ICharacterBuilder DoDeed(string deed) { _character.EvilDeeds.Add(deed); return this; }
        public Character GetResult() { return _character; }
    }

    public class Director
    {
        public void ConstructDreamHero(ICharacterBuilder builder)
        {
            builder.SetHeight(185)

```

```

        .SetBuild("атлетична")
        .SetHairColor("світле")
        .SetEyeColor("блакитні")
        .SetClothing("сяючі лицарські обладунки")
        .SetInventory("меч світла, щит, зілля зцілення")
        .DoDeed("врятував королівство")
        .DoDeed("допоміг бідним");
    }

    public void ConstructNemesis(ICharacterBuilder builder)
    {
        builder.SetHeight(205)
            .SetBuild("кремезна")
            .SetHairColor("лисий")
            .SetEyeColor("червоні")
            .SetClothing("темний плащ із капюшоном")
            .SetInventory("посох темряви, Отрута")
            .DoDeed("викрав принцесу")
            .DoDeed("зруйнував міст");
    }
}

class Program
{
    static void Main(string[] args)
    {
        Console.OutputEncoding = System.Text.Encoding.UTF8;

        Director director = new Director();

        ICharacterBuilder heroBuilder = new HeroBuilder();
        director.ConstructDreamHero(heroBuilder);
        Character hero = heroBuilder.GetResult();
        hero.ShowInfo("Герой");

        ICharacterBuilder enemyBuilder = new EnemyBuilder();
        director.ConstructNemesis(enemyBuilder);
        Character enemy = enemyBuilder.GetResult();
        enemy.ShowInfo("Ворог");

        Console.ReadKey();
    }
}

```

Результат:

		Мазурова І. В.			ДУ «Житомирська політехніка».26.F2.06.000 – Лр.2	Арк.
		Левківський В. Л.				
Змн.	Арк.	№ докум.	Підпис	Дата		18

```
--- Герой ---
Зріст: 185 см, Статура: атлетична
Волосся: світле, Очі: блакитні
Одяг: сяючі лицарські обладунки
Інвентар: меч світла, щит, зілля зцілення
Добрі справи: врятував королівство, допоміг бідним

--- Ворог ---
Зріст: 205 см, Статура: кремезна
Волосся: лисий, Очі: червоні
Одяг: темний плащ із капюшоном
Інвентар: посох темряви, Отрута
Злі справи: викрав принцесу, зруйнував міст
```

Рис. 5.1. Результат роботи патерну «Будівельник» при створенні персонажів

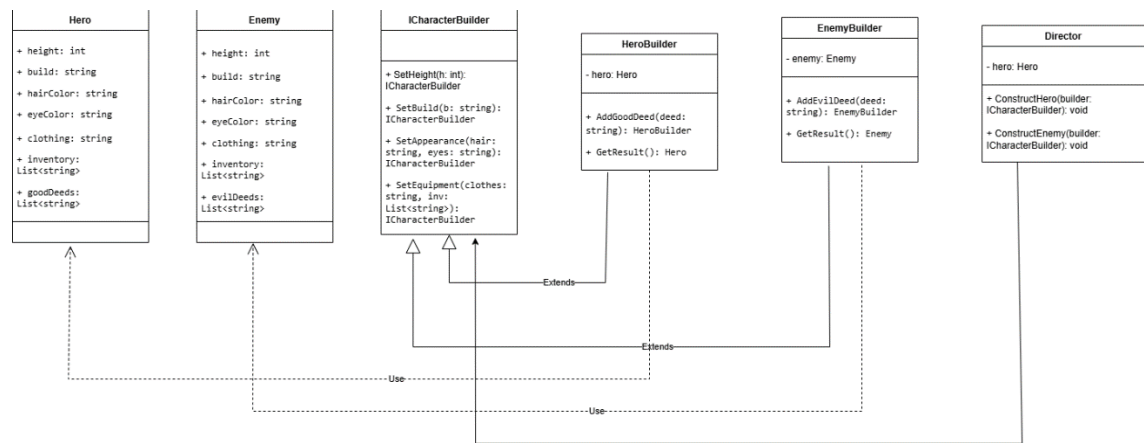


Рис. 5.2. UML-діаграма класів

Висновок: під час виконання лабораторної роботи я навчилася реалізовувати породжувальні шаблони проєктування.